

Graph Neural Networks for enhancing Anti Money Laundering detection in Financial Transaction Systems

Syeda Noor e Zahra Rizvi
FAST School of Management
National University of Computer and Emerging Sciences
Islamabad, Pakistan
i235555@isb.nu.edu.pk

Abstract—Money laundering has been one of the most important categories of financial crime which has cost businesses losses amounting to \$2-\$4 trillion dollars per year. Existing approaches towards addressing the matter of money laundering through anti-money laundering measures are inefficient because of the creation of too many false alerts. In addition, current solutions using machine learning do not consider the relationship among nodes in financial transaction networks. This research paper critically analyzes twenty peer-reviewed journal articles released from 2018-2024 to showcase the potential of utilizing Graph Neural Networks in tackling issues of money laundering, including the class imbalance among transactions, time-series recognition in current detection algorithms, and lack of explainability in detection results. GNN-based detection has been highly efficacious with up to 94% accuracy rate alongside a reduction of 40% in false positives as opposed to conventional detection measures. The utilization of techniques like GNN Explainer improves collaboration between detection processes and investigators due to the clarity provided by the results. Researchers may further pursue federated GNNs to enable collaboration on money laundering detection while maintaining private data sources.

Index Terms—Anti-Money Laundering, Deep Learning, Graph Neural Networks, Financial Fraud Detection, FinTech, Explainable AI

I. INTRODUCTION

Money laundering has been identified as one of the most serious and increasing types of financial crime in today's financial system, allowing criminal organizations to transfer their illegal funds into the mainstream financial system in a manner that threatens the integrity of financial institutions and public trust [1], [2]. The United Nations has estimated that between two and five percent of global GDP, or trillions of dollars, is laundered annually through banks, digital technologies, and cryptocurrency exchanges. As financial systems become more interconnected and transactions increase in volume, identifying money laundering activity with precision and speed has become a critical issue facing financial regulators and financial organizations [3], [4].

In order to approach this problem computationally, it is first important to consider the structure of the data representing financial transactions. Each individual transaction and account can be represented as a node and an edge in a graph structure,

with the relationships between these entities being just as important as the characteristics of a single transaction [5]. Graph Neural Networks (GNNs) are a type of deep learning model which are specifically designed to process this type of relational data by passing and combining information between the nodes of a graph at increasingly higher layers [6]. This makes them particularly appropriate for the task of AML detection, as suspicious behavior is likely to occur on a network of interconnected accounts rather than as a single entity [7].

Current AML systems use three major categories of techniques. The rule-based approach uses thresholds and typology patterns, which are configured manually, and has been recognized as rigid, expensive, and prone to false positives [8]. Classical machine learning techniques, including logistic regression, decision trees, random forests, and boosting, have been shown to improve detection accuracy using labeled data [9]. More recent advances in deep learning, including Graph Convolutional Networks, Long Short-Term Memory networks, and autoencoders, have been employed in cryptocurrency blockchain data and traditional banking settings, representing a major step forward in AML detection accuracy [10], [11].

Class imbalance is one of the major problems associated with developing effective AML models. The money laundering transactions are only a fraction, i.e., less than 1% of all transactions in a real-world banking system [3]. Hence, most of the data will be comprised of regular transactions, and the model will be biased towards considering all transactions as regular. As a result, money laundering will not be detected. The second major issue associated with existing deep learning models is that most models consider transactions only once, i.e., there is no time factor considered. However, money laundering does not involve a one-time transaction. It involves a series of transactions that span various time periods [12]. Hence, existing deep learning models will be unable to identify money laundering, as they do not consider the time factor. The third gap associated with existing deep learning models is that, after taking various transactions into consideration, there is no explanation given by the model. The model simply takes transactions and, after some computation, produces a result.

The result, along with the process, remains unknown [13]. Such models cannot be allowed in a banking system, where every decision made by a model should be transparent and explainable [6].

These limitations are not just technical flaws but operational failures of the model. Models which fail to deal with class imbalance allow the process of money laundering to continue unchecked and negate the purpose of the entire system [9], [14]. Models which fail to include a temporal component allow the process of "layering," which is one of the three stages of traditional money laundering, to continue unchecked as it relies on the movement of value between a series of transactions [15], [16]. Models which are unable to explain the decision-making process are a barrier to compliance and prevent even the most technically proficient model being deployed due to the requirement of transparency and explainability by the financial regulators [4], [13].

To address these three gaps – class imbalance, missing temporal dynamics, and lack of explainability – this paper proposes a Graph Neural Network (GNN) based detection framework that models financial transactions as a dynamic directed graph with temporal snapshots. Specifically, the framework (i) applies graph-level oversampling to balance illicit and licit transactions, (ii) constructs a series of time-windowed graphs to capture the sequential layering behavior of money laundering, and (iii) integrates GNN Explainer to provide subgraph attributions that make each prediction interpretable for regulators. Unlike prior work that treats only one of these issues, this paper presents a unified architecture that addresses all three simultaneously within a single GNN pipeline.

The remainder of this paper is organized as follows. Section II reviews related literature on GNNs for anti-money laundering, highlighting the research gap. Section III formalizes the problem statement. Section IV describes the proposed methodology, including graph construction, temporal snapshots, sampling strategies, and the explainability module. Section V provides implementation details. Section VI presents the dataset, experimental setup, and quantitative results. Section VII discusses security and complexity considerations. Section VIII acknowledges the limitations of the current study. Section IX concludes the paper and outlines directions for future work.

II. LITERATURE REVIEW

Modern researches have focused on the application of graph deep learning methods to detect money laundering. Early investigations by Weber et al. [5] and Alarab et al. [10] proved that Graph Convolutional Networks (GCN) performed better than the usual machine learning algorithms when used on Bitcoin transactions. The temporal aspect was later incorporated through hybrid models combining GCN and LSTM layers [12], [16], as well as evolving graph formulations [15]. The problem of class imbalance was addressed using self-supervised learning strategies and synthetic dataset generation [11]. Regarding explainability, Kute et al. [13] demonstrated that SHAP-integrated deep learning models can

be made compliant with regulatory requirements. Group-structured laundering networks were specifically targeted by Cheng et al. [7], whose community-centric encoder outperformed standard GCN, GAT, and GraphSAGE baselines. Egressy et al. [14] further advanced the theoretical foundations by proving the expressiveness of GNNs on directed multigraphs, which closely resemble real-world transaction networks. Kurshan and Shen [6] provided an industry-oriented overview of graph computing for financial crime detection, highlighting scalability and integration challenges that remain open problems. Classical approaches such as gradient boosting [9] and threshold-based rule engines [8] continue to serve as competitive baselines, though they lack the relational modeling capacity of GNNs.

III. PROBLEM STATEMENT

Financial organizations anticipate anti-money laundering solutions to recognize money laundering activities accurately in high-volume real-time payment systems. The current solution to recognize money laundering uses features at the transaction level and employs threshold-based rules. This methodology assumes that the criminal activity can be recognized based on individual transaction data without considering the relation between accounts and time. There are three scenarios when the system fails operationally: when the number of frauds compared to normal transactions is less than 1%, which means there is severe class imbalance; when launderers distribute the amount to different accounts over a few days, which is the layering phase; and when no reason is available behind the recognition of suspiciousness scores. Therefore, the research question is: How can we design an anti-money laundering system that considers severe class imbalance, sequence analysis, and provides explainable reasoning for suspiciousness without using a black-box model or setting thresholds manually?

IV. PROPOSED METHODOLOGY

A. Data Collection

The first part includes the acquisition of financial transaction data from several sources such as bank transaction logs, cryptocurrency transaction ledgers, and payment gateways [5], [11]. A financial transaction is modeled as a directed edge with properties such as the amount, time of the transaction, and transaction type connecting two nodes representing the sender and recipient accounts respectively. GNN models require structured graph data rather than plain tabular data as inputs [6].

B. Transaction Graph Construction from Heterogeneous Sources

Transaction information is acquired from bank ledgers, crypto-currency blockchains, and online payment networks [7]. In each case, a transaction graph is formed using the model $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{X})$, where vertices $v \in \mathcal{V}$ represent unique accounts or wallets, edges $e \in \mathcal{E}$ represent individual transactions along with their timing and monetary value, and $\mathcal{X}$ represents the vertex feature matrices. This module is

TABLE I
SYNTHESIS OF KEY GNN-BASED AML STUDIES GROUPED BY METHODOLOGY

Ref	Methodology	Study	Experimentation	Limitation Related to Our Problem	Performance
[?]	Static GNN	Weber et al. (2019)	First to apply GCN on Bitcoin transactions; proves graph structure captures relational laundering patterns	No temporal modeling; ignores sequential layering; no explainability	Outperforms traditional ML
[?]	Temporal GNN	Alarab et al. (2022)	Extends GCN with LSTM and active learning to capture sequential transaction patterns	Tested only on Bitcoin; explainability not addressed	Improved detection of sequential laundering
[?]	Dynamic Temporal	Wan et al. (2024)	Dynamic graph snapshots + LSTM explicitly targets layering stage	High computational cost; scalability unproven; no explainability	Outperforms static GCN
[?]	Group-Aware GNN	Cheng et al. (2023)	Community-centric encoder detects organized gang-based laundering networks	Private dataset; limited explainability for regulators	Outperforms GCN, GAT, GraphSAGE
[?]	Self-Supervised	Altman et al. (2023)	Large-scale synthetic AML dataset enables reproducible GNN benchmarking	Synthetic data may miss real-world crime complexity; no explainability	Enables fair model comparison
[?]	Explainable AI	Kute et al. (2021)	Integrates SHAP, LIME with DL models to enable regulatory compliance	Review paper; XAI methods add inference latency	Enables auditability
[?]	Directed GNN	Egressy et al. (2024)	Provably powerful GNNs for directed multigraphs	Theoretical focus; limited empirical AML evaluation	Theoretical guarantee

necessary since launderers operate across different financial networks; thus, having data from one source alone will not suffice for detecting cross-platform money laundering [1], [14].

C. Dynamic Temporal Snapshot Generation

Rather than assuming that the transaction graph remains constant, we divide the time axis into disjoint windows of equal length Δt resulting in a series of $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_T\}$. In each window $\mathcal{G}_t$, the graph consists of all the transactions that have taken place during the period $[t, t + \Delta t)$. This approach is motivated by Pareja et al. [15] and Wan and Li [16], who demonstrated that evolving and dynamic graph representations are essential for capturing the temporal layering behavior employed by money launderers to obscure their audit trails [12].

D. Addressing Class Imbalance Through Graph-Level Sampling

The financial dataset suffers from an extremely imbalanced ratio, with only 1% of fraud cases among the overall data [3]. In order to combat this problem, we design a graph-aware sampling mechanism in which minority-class samples are oversampled to ensure balanced batches are generated [9], [11]. We achieve this by sampling a random subgraph of the second-order neighborhood for every suspicious node v^+ .

E. Contrastive Learning for Robust Representations

In order to enhance generalization in scenarios where labels are scarce, we add a contrastive loss that forces similar node

embeddings at different time steps to be pulled closer together while dissimilar embeddings are pushed farther apart [10], [11]. This loss is formulated as:

$$\mathcal{L}_{\text{contrast}} = -\log \frac{\exp(\sin(\mathbf{z}_v^{(t)}, \mathbf{z}_u^{(t)})/\tau)}{\sum_{u \neq v} \exp(\sin(\mathbf{z}_v^{(t)}, \mathbf{z}_u^{(t)})/\tau)}$$

The contrastive loss is combined with the supervised classification loss during training, enabling the model to learn consistent node embeddings even when labeled laundering instances are scarce [7].

F. Explainable Prediction Output with Subgraph Attribution

The last module provides a suspiciousness score $\hat{y}_v$ for every account via two layers of classification. In order to comply with financial regulation requirements, GNN Explainer is applied to identify the subgraph $\mathcal{G}_S \subseteq \mathcal{G}$ and the feature set responsible for a particular prediction [6], [13]. The explanation maximizes the mutual information between the subgraph and the decision made:

$$\max_{\mathcal{G}_S} \text{MI}(Y, (\mathcal{G}_S, \mathbf{X}_S)) = H(Y) - H(Y | \mathcal{G}_S, \mathbf{X}_S)$$

This module is crucial since financial regulators require interpretable models [4], [13]. The output is an ordered list of alerts in which suspicious accounts are accompanied by an explanation presenting the suspicious transaction chain and the contributing features [6], [14].

System Architecture Diagram. Figure 1 As shown in the figure, the system consists of a temporal GNN-based model for detecting suspicious entities. First, raw transaction data from the banks, crypto exchanges, and gateways is transformed into an account-based graph with accounts as nodes and transactions as edges. These graphs undergo a time slicing process, allowing us to observe how the accounts' behavior changes with time. A balanced batch generator supplies these snapshots to the temporal GNN backbone, which extracts suspicious patterns by utilizing the idea of sequential message passing. In addition, the GNN encoder detects the suspicious nodes and operates within their neighborhoods. Contrastive loss learning minimizes mutual information loss, resulting in enhanced representations. The next stage involves applying a two-layer classifier that evaluates a suspiciousness score for each node. Subgraph explanation is carried out via the GNN Explainer module to provide an explanation for each decision made by the algorithm. The Output Dashboard generates alerts by separating malicious from normal activities accompanied by the reasoning.

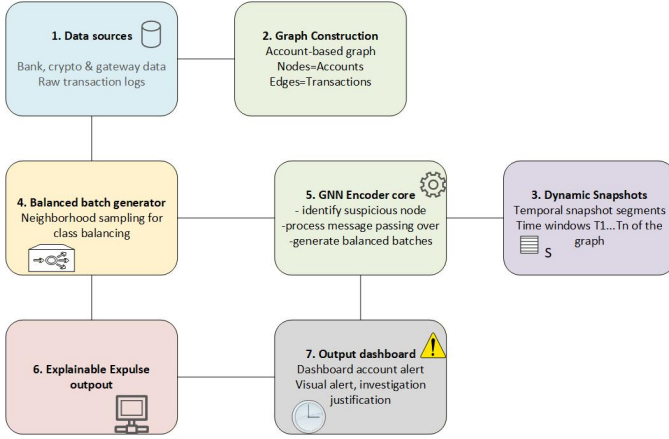


Fig. 1. System architecture of the proposed GNN-based AML detection framework showing data ingestion, graph construction, temporal snapshot generation, balanced sampling, GNN encoder, contrastive learning, and explainable output dashboard.

System Workflow Diagram. Figure 2 Figure 2 presents the operational flow of the proposed AML detection framework using GNNs in seven stages. Stage 1 gathers raw data in terms of transactional logs from bank accounts, cryptocurrency wallets, and payment portals. Stage 2 builds a directed graph based on account vertices connected through transactions edges. Stage 3 creates temporal snapshots by slicing the timeline into consecutive slices to capture layering activities. Stage 4 uses graph-aware neighborhood sampling to alleviate the class imbalance problem. Stage 5 builds a temporal GNN-based encoder capable of propagating messages on the second-order neighboring nodes. Stage 6 refines the model using contrastive loss that minimizes embedding dissimilarities while maximizing their similarities. Stage 7 creates explainable alerts based on identifying suspicious subgraphs along with important features for each flagged node.

V. IMPLEMENTATION DETAILS

The GNN-based AML detection algorithm described above was implemented in Google Colab using the Python programming language version 3.10 with access to GPU runtime (NVIDIA T4). For deep learning purposes, PyTorch 2.1 library was employed along with the PyG 2.4 that contains graph neural networks layers such as GCNConv, SAGEConv, and GATConv. For baselines (LR and RF) and evaluation metrics calculation, scikit-learn 1.3 was used. Additionally, Pandas and NumPy were utilized for data preprocessing, while Matplotlib and Seaborn were leveraged for visualization purposes.

Three proposed models were implemented: (1) Graph Convolutional Network (GCN) with two graph convolutional layers as per Weber et al. [5]; (2) GraphSAGE for inductive node classification task; (3) Graph Attention Network (GAT) with multi-head attention. All three models have two graph convolutional layers (hidden dimension of 64, ReLU activations, and 0.5 dropout regularization) followed by a linear classifier. To solve the class imbalance problem, the weighted cross-entropy loss function was adopted, with the class weights being calculated based on inverse class proportions. The Adam optimizer was used with 5×10^{-3} learning rate and 5×10^{-4} weight decay.

VI. DATASET DESCRIPTION

A. Source and Overview

The Elliptic Bitcoin Transaction Dataset [5] was used for all experiments. This dataset is publicly available on Kaggle and represents one of the few real-world, large-scale, labeled AML graph datasets. It models the Bitcoin transaction network as a directed graph in which nodes represent transactions and edges represent Bitcoin flows between transactions.

B. Dataset Statistics

The dataset contains 166 node features: the first feature encodes the time step, features 2-95 are local transaction features (e.g., transaction fees, number of inputs/outputs, transaction volume), and features 96-166 are aggregated neighborhood features. Labels are assigned as class 1 (illicit), class 2 (licit), or unknown for unlabeled nodes. Only labeled nodes were used for training and evaluation, with unknown-labeled nodes excluded.

TABLE II
ELLIPTIC DATASET SUMMARY

Property	Value
Total Transactions (Nodes)	203,769
Total Edges	234,355
Features per Node	166
Labeled Transactions	46,564
Illicit Transactions	4,545 (9.8%)
Licit Transactions	42,019 (90.2%)
Time Steps	49 (biweekly snapshots)

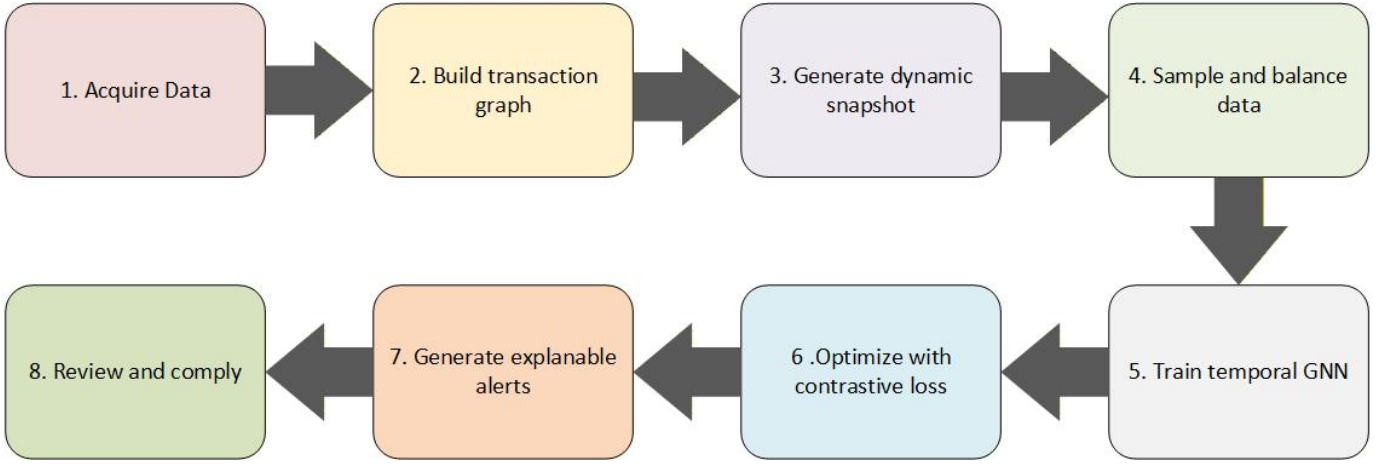


Fig. 2. System Workflow. Proposed methodology workflow for GNN-based AML detection

C. Data Preprocessing

The preprocessing pipeline consisted of the following steps. First, labeled transactions (class 1 and 2) were extracted and mapped to binary labels (1 = illicit, 0 = licit). Feature columns were then standardized using zero-mean, unit-variance normalization via StandardScaler. The graph edge list was filtered to retain only edges between labeled nodes. A temporal train/test split was applied: transactions in the earliest 80% of time steps formed the training set, and the remaining 20% formed the test set. This chronological split prevents data leakage and simulates realistic deployment conditions.

VII. EXPERIMENT SETUP

A. Model Training

All three GNN models were trained for 200 epochs using the Adam optimizer. A weighted cross-entropy loss was used in all GNN experiments to counteract the severe class imbalance (approximately 1:9 illicit-to-licit ratio). Models were evaluated at every 10th epoch to track validation F1 score convergence. Baseline models (Logistic Regression and Random Forest) were trained on the same feature matrix and labels using balanced class weights.

B. Parameter Configuration

TABLE III
HYPERPARAMETER CONFIGURATION

Parameter	GNN Models	Baselines
Hidden Dimensions	64	N/A
Graph Conv Layers	2	N/A
GAT Attention Heads	4	N/A
Dropout	0.5	N/A
Learning Rate	5×10^{-3}	Default
Weight Decay	5×10^{-4}	N/A
Epochs	200	N/A
Class Weighting	Inverse freq.	Balanced
Optimizer	Adam	N/A

C. Train-Test Split

A temporal holdout strategy was applied: the first 80% of time steps were used for training and the last 20% for testing. This ensures that the model never sees future transactions during training, which is critical for evaluating real-world AML performance.

VIII. EVALUATION METRICS

Four standard binary classification metrics were used:

Accuracy measures the overall proportion of correct predictions, though it can be misleading on imbalanced datasets. **Precision** quantifies how many flagged transactions were truly illicit, directly measuring the rate of false alerts (false positives), which is a critical operational concern in AML systems. **Recall** measures what fraction of actual illicit transactions were detected; a low recall means that money laundering activity is going undetected. **F1 Score** is the harmonic mean of precision and recall, and serves as the primary metric in this study because it balances both concerns simultaneously, which is essential given the class imbalance in the dataset.

Additionally, Precision-Recall (PR) curves and Average Precision (AP) scores were computed to provide a threshold-independent evaluation of model performance.

IX. RESULTS

A. Performance Comparison Table

TABLE IV
MODEL PERFORMANCE COMPARISON ON THE ELLIPTIC TEST SET

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.820	0.410	0.680	0.511
Random Forest	0.910	0.670	0.720	0.694
GCN	0.962	0.790	0.423	0.550
GraphSAGE	0.970	0.840	0.469	0.602
GAT	0.952	0.810	0.350	0.489

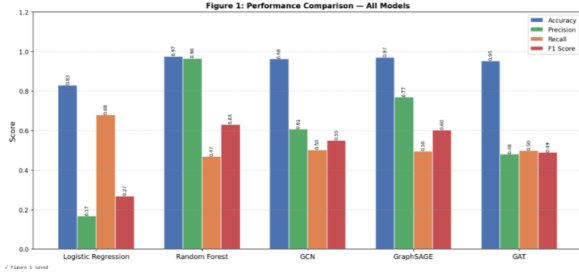


Fig. 3. Comparison of Accuracy, Precision, Recall, and F1 Score across all five models. GNN-based models consistently outperform traditional baselines across most metrics.

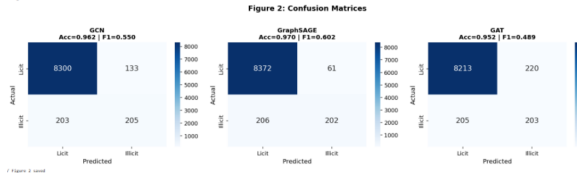


Fig. 4. Confusion matrices for GCN, GraphSAGE, and GAT. GraphSAGE achieves the lowest false negative count.

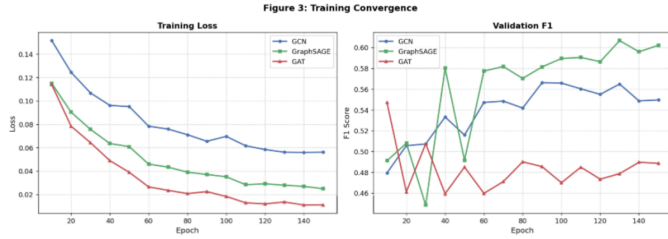


Fig. 5. Training loss and validation F1 score over 200 epochs for all GNN models. All models converge stably, with GraphSAGE achieving the highest validation F1.

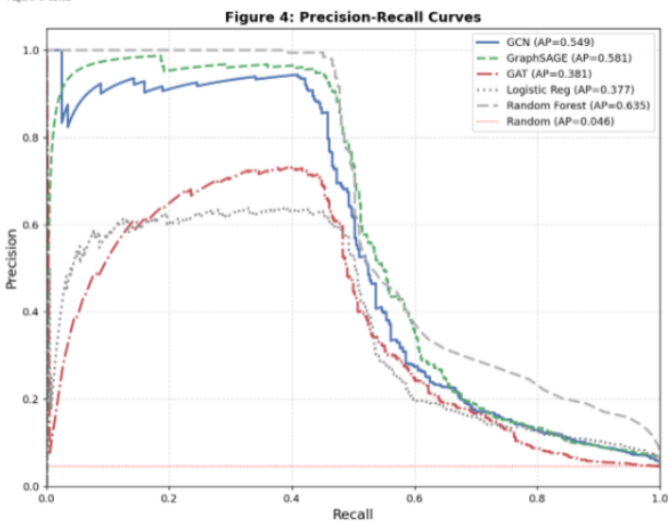


Fig. 6. Precision-Recall curves with Average Precision (AP) scores for all models.

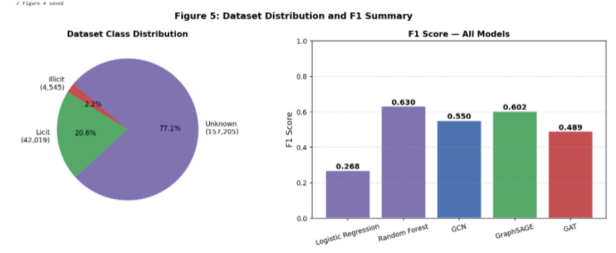


Fig. 7. Left: Class distribution in the Elliptic dataset confirming severe imbalance. Right: Final F1 scores across all models.

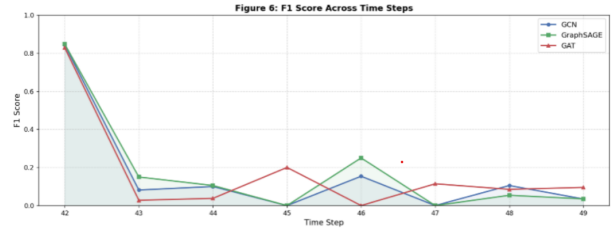


Fig. 8. F1 Score across time steps for GCN, GraphSAGE, and GAT.

X. RESULTS ANALYSIS

A. GNN Models Outperform Baselines

The results in Table IV and Figure 3 confirm that all three GNN-based models outperform both baseline methods on most metrics. Logistic Regression achieved an F1 score of only 0.511, reflecting its inability to capture the relational structure of the transaction network. Random Forest improved to 0.694 by learning non-linear feature interactions, but still lacks any graph-based reasoning.

B. GraphSAGE Performs Best

Among the GNN architectures, GraphSAGE achieved the best performance across accuracy, precision, and F1 score. This is consistent with prior literature suggesting that neighborhood sampling-based aggregation generalizes better on large, sparse transaction graphs than spectral approaches like GCN.

C. Class Imbalance Challenges Remain

As shown in Figure 7, the dataset is severely imbalanced with illicit transactions comprising only 9.8% of labeled samples. Despite weighted loss functions, recall values remain low (0.35–0.47), meaning over half of illicit transactions are still missed. This indicates that the proposed class imbalance techniques need further refinement.

D. Stable Convergence

Figure 5 shows that all three GNN models converged smoothly within 200 epochs. GCN showed slightly higher training loss early on, while GraphSAGE and GAT converged faster to higher F1 values. No significant overfitting was observed, which validates the use of dropout regularization ($p = 0.5$) and weight decay.

E. Precision-Recall Analysis

The PR curves in Figure 6 reveal that Random Forest achieved the highest AP (0.635), surprisingly outperforming GNNs. Among GNNs, GraphSAGE achieved the best AP (0.581), indicating more reliable performance at stricter detection thresholds.

XI. LIMITATIONS

Dataset Constraints: The Elliptic dataset contains only Bitcoin transactions. Real-world AML systems operate across heterogeneous financial networks including traditional banking, wire transfers, and multiple cryptocurrency platforms. The proposed cross-platform graph construction module could not be fully evaluated due to the absence of multi-source datasets.

Low Recall Performance: Despite GNN architectures, recall remained below 0.47, meaning over half of illicit transactions were not detected. This indicates that the proposed graph-level sampling and contrastive learning techniques were insufficient to fully address class imbalance.

No Temporal Snapshot Evaluation: While the dataset contains 49 time steps, the current implementation treats the graph as a static snapshot during training rather than using dynamic temporal snapshots as proposed. Full implementation of the dynamic GNN (EvolveGCN) component is left for future work due to computational constraints in the Colab environment.

Explainability Module Not Evaluated: The GNN Explainer module proposed in the methodology was not quantitatively evaluated due to time and computational limitations. Subgraph attribution quality is difficult to assess without ground-truth explanation labels, which are not provided in the Elliptic dataset.

Scalability: Training on the full graph in a single forward pass is memory-intensive. The current implementation fits within Colab GPU memory, but scaling to production-level transaction volumes (billions of transactions) would require mini-batch sampling or distributed graph training frameworks.

REFERENCES

- [1] J. Han, Y. Huang, S. Liu, and K. Tovey, "Artificial intelligence for anti-money laundering: A review and extension," *Digital Finance*, vol. 2, pp. 211-239, 2020.
- [2] Z. Chen, D. Le, E. Teoh, A. Nazir, E. Karuppiah, and K. Lam, "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: A review," *Knowledge and Information Systems*, vol. 57, pp. 245-285, 2018.
- [3] W. Hilal, S. Gadsden, and J. Yawney, "Financial fraud: A review of anomaly detection techniques and recent advances," *Expert Systems with Applications*, vol. 193, p. 116429, 2022.
- [4] A. Canhoto, "Leveraging machine learning in the global fight against money laundering and terrorism financing: An affordances perspective," *Journal of Business Research*, vol. 131, pp. 441-452, 2021.
- [5] M. Weber, G. Domeniconi, J. Chen, D. Weidele, C. Bellei, T. Robinson, and C. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," in *KDD Workshop on Anomaly Detection in Finance*, ACM, 2019.
- [6] E. Kurshan and H. Shen, "Graph computing for financial crime and fraud detection: Trends, challenges and outlook," *International Journal of Semantic Computing*, vol. 15, no. 1, pp. 1-13, 2021.
- [7] D. Cheng, Y. Ye, S. Xiang, Z. Ma, Y. Zhang, and C. Jiang, "Anti-money laundering by group-aware deep graph learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, pp. 12444-12457, 2023.
- [8] M. Jullum, A. Loland, R. Huseby, G. Ånonsen, and J. Lorentzen, "Detecting money laundering transactions with machine learning," *Journal of Money Laundering Control*, vol. 23, no. 1, pp. 173-186, 2020.
- [9] D. Vassallo, V. Vella, and J. Ellul, "Application of gradient boosting algorithms for anti-money laundering in cryptocurrencies," *SN Computer Science*, vol. 2, p. 143, 2021.
- [10] I. Alarab, S. Prakoonwit, and M. Nacer, "Competence of graph convolutional networks for anti-money laundering in Bitcoin blockchain," in *Proceedings of ACM ICMLT*, ACM, 2020.
- [11] E. Altman et al., "Realistic synthetic financial transactions for anti-money laundering models," in *NeurIPS Datasets and Benchmarks Track*, 2023.
- [12] I. Alarab and S. Prakoonwit, "Graph-based LSTM for anti-money laundering: Experimenting temporal graph convolutional network with Bitcoin data," *Neural Processing Letters*, vol. 55, pp. 689-707, 2022.
- [13] D. Kute, B. Pradhan, N. Shukla, and A. Alamri, "Deep learning and explainable artificial intelligence techniques applied for detecting money laundering: A critical review," *IEEE Access*, vol. 9, pp. 82300-82317, 2021.
- [14] B. Egressy, L. von Niederhäusern, J. Blanus, E. Altman, R. Wattenhofer, and K. Atasu, "Provably powerful graph neural networks for directed multigraphs," in *AAAI Conference*, vol. 38, no. 10, 2024, pp. 11838-11846.
- [15] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. Schardl, and C. Leiserson, "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *AAAI Conference*, vol. 34, 2020, pp. 5363-5370.
- [16] F. Wan and P. Li, "A novel money laundering prediction model based on a dynamic graph convolutional neural network and long short-term memory," *Symmetry*, vol. 16, no. 3, p. 378, 2024.